Module 1
Chapter 1: Introduction

- **What is Operating System?** : An Operating system is a program or software that manages the computer hardware.
- It also provides a basis for application programs and acts as an intermediary between computer user and the computer hardware.

Examples: Windows, Linux, Android, IoS, etc.

- **The OS has 3 main functions:**

1. Manages the computer's resource such as CPU, memory, etc.
2. Establish a user interface, and
3. Execute and provide services for application software.

1.1 What Operating System Do:

- A computer system can be divided roughly into **4 components:**

a) Hardware,
b) Operating system,
c) Application programs, and
d) The users.

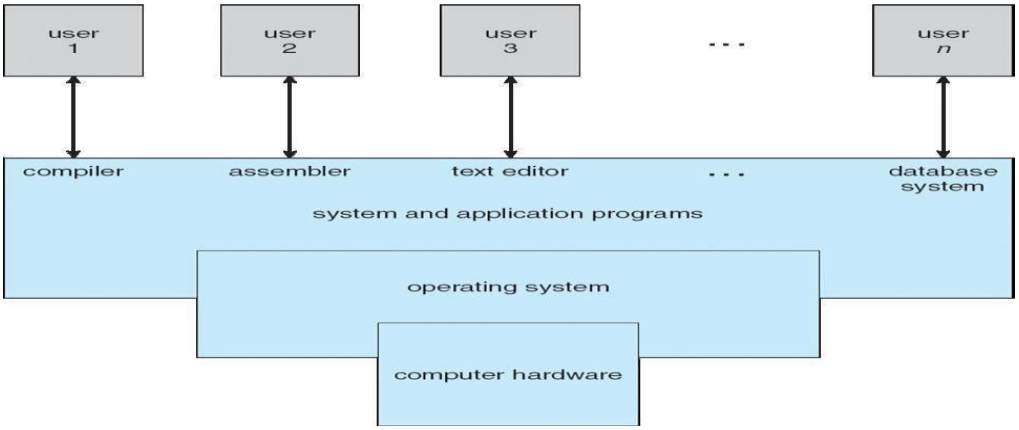


Figure 1.1: Abstract View of the Components of a computer System.

a) **Hardware**: the central processing unit (CPU), the memory, and input/output devices- provide the basic computing resources for the system.

b) **The operating system**: controls and coordinates the use of the hardware among the various application programs for the various users.

c) **The application programs**: such as word processors, spreadsheets, compliers, and web browsers- defines the way in which these resources are used to solve user's computing problems.

1.1.1 User View:

- The user's view of computer system varies according to the interface being used.

1) Most computer users uses **PC**, such systems are designed for one user to monopolize its resources.
- The goal is to maximize the work that the user is performing.
- In this case, the OS is designed for ease of use.

2) A user sits at a terminal connected to **mainframe or minicomputer**. Other users are accessing the same computer through other terminals.
- These users share resources and may exchange information.
- Here, OS is designed to maximize resource utilization.

3) User sits at **workstations** connected to networks of other workstations and servers.
- Users have dedicated resources and also share resources such as networking and servers.
- OS is designed to compromise between individual usability and resource utilization.

4) Most of the **handheld computers** are standalone units for individual users.
- Some are connected to networks, either directly by wire or through wireless modems and networking.
- OS is designed for individual usability, but performance per amount of battery life is important as well.

5) Some computers have **little or no user view**.
- For e.g., embedded computers in home devices and automobiles may have numeric keypads and may turn indicator lights on or off to show status.
- OS are designed primarily to run without user intervention.

1.1.2 System View:

- From computer's point of view, the OS is the program most intimately involved with the hardware.
- In this context, we can view an OS as **resource allocation.**
- A computer system has many resources that may be required to solve a problem: CPU time, memory space, file-storage space, I/O devices, and so on.
- The OS acts as the manager of these resources. Facing numerous and possibly conflicts requests of resources, the OS must decide how to allocate them to specific programs and users so that it can operate the computer system efficiently.

- A slightly different view of an OS emphasizes the need to control the various I/O devices and user programs.
- An OS is a control program.
- A control program manages the execution of user programs to prevent errors and improper use of the computer. It is especially concerned with operation and control of I/Odevices.

1.4 Operating System Structure:

- An OS provides the environment within which programs are executed.
- One of the most important aspects of OS is the ability to multi-program.
- **Multiprogramming** increases CPU utilization by organizing jobs (code and data) so that CPU always has one to execute.
- The idea is as follows:
  - The OS keeps several jobs in memory simultaneously. This set of jobs can be a subset of jobs kept in the jobs pool. Job pool contains all jobs that enter the system
  - OS picks a job and begin to execute one job via **job scheduling.**
  - The job may have to wait for some task (for I/O for example), then OS switches to another job and execute.
- Eventually, the first job finishes waiting and gets the CPU back. As long as at least one job needs to execute, the CPU is never idle.
- Multi-programmed systems provide an environment in which the various systems resources are utilized effectively, but they do not provide for user interactions with computer system.

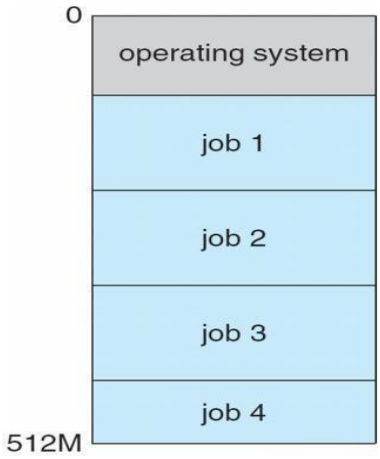


Figure 1.5 Memory layout of Multiprogramming System.

- **Timesharing or (multitasking)** is logical extension of multi-programming in which CPU switches jobs so frequently that users can interact with each job while it is running, and creating interactive computing.
- Response time should be < 1 second.
- Each user has at least one program executing in memory. The program loaded into memory and executing is called **process**.
- If several jobs ready to be brought into memory at the same time, and if there is not enough room for all of them, then system must choose among them. Making this decision is **Job scheduling**.
- If processes don't fit in memory, swapping moves them in and out to run.
- Virtual memory is a technique that allows execution of processes that is not completely in memory.

1.5 Operating System Operations:

- Modern computers are interrupt driven. If there are no processes to execute, no I/O devices to service, and no users to whom to respond, an OS will sit quietly, waiting for something to happen.
- Events are almost signaled by the occurrence of an interrupt or a trap.
- A trap (or a exception) is a software-generated interrupt caused either by an error or by a specific request from a user program that an OS service be performed.

1.5.1 Dual Mode Operation:

- In order to ensure the proper execution of the OS, we must be able to distinguish between the execution of OS code and user-defined code.
- **There are two modes of operation:**

1. User mode and
2. Kernel mode (Supervisor mode, system mode or privileged mode).

- A bit, called the mode-bit, is added to the hardware of the computer to indicate the current mode: **Kernel (0) or User (1).**
- With a mode bit, users are able to distinguish between a task that is executed in user mode or kernel mode.
- When the computer system is executing on behalf of user application, the system is in user mode.
- When a user application requests a service from OS(via system call), it must transit from user to kernel mode to fulfill the request.

- Whenever trap or interrupt occurs, the hardware switches from user mode to kernel mode.
- In kernel mode, the OS gains control of the system. The system always switches to user mode before passing control to user program.
- The dual mode operations provide the protection to OS from unauthorized users. The hardware allows privileged instructions to be executed only in kernel mode.
- If attempt is made to execute a privileged instruction in user mode, the hardware does not execute the instruction but rather treats it as illegal and traps it to the OS.

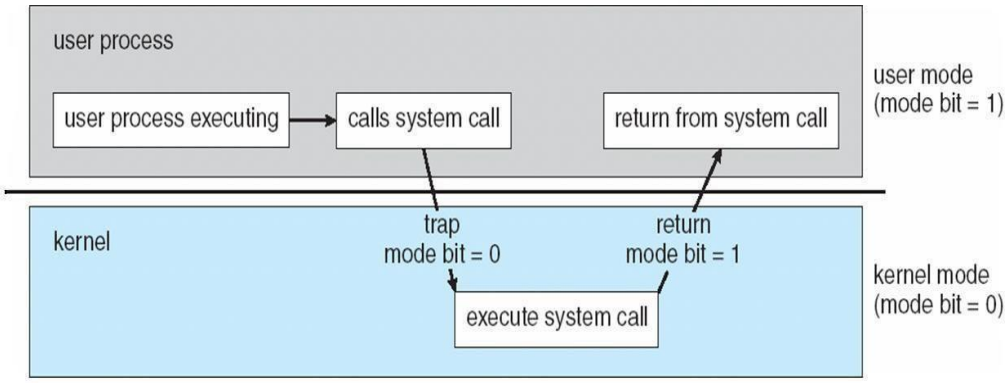


Figure 1.6: Transition from user to kernel mode.

- Software may trigger an interrupt by executing a special operation called a system call.
- System call provides the means for user program to ask the OS to perform tasks reserved for OS on the user's program behalf.
- When system call is executed, it is treated by the hardware as a s software interrupt.

1.5.2 Timer:

- Timer is used to prevent infinite loop / process hogging resources.
- Timer can be set to interrupt computer after specific period.
- Operating system sets the counter. Every time the clock ticks, counter is decremented.
- Before turning over control to the user, the OS ensures that the timer is set to interrupt.
- If the timer interrupt's, control transfers automatically to OS, which may treat interrupt as fatal error or may give program more time.
- Thus, we can use timer to prevent user program from running too long.

Chapter 2: System Structures

2.1 Operating-System Services:

- An operating system provides an environment for the execution of programs.
- It provides certain services to programs and to the users of those programs.
- **OS provide services for the users of the system**, including:

1. **User Interfaces** - Means by which users can issue commands to the system. All OS have user interface **(UI).**
   - This interface can take several forms.
   - One of them is a **command line interface (CLI),** which uses text commands and a method for entering them.
   - Another is **batch interface**, in which commands and directories to control those commands are entered into files, and those files are executed.
   - Most commonly **graphical user interface (GUI)** is used. In GUI systems- windows with pointing device to get inputs and keyboard to enter the text.

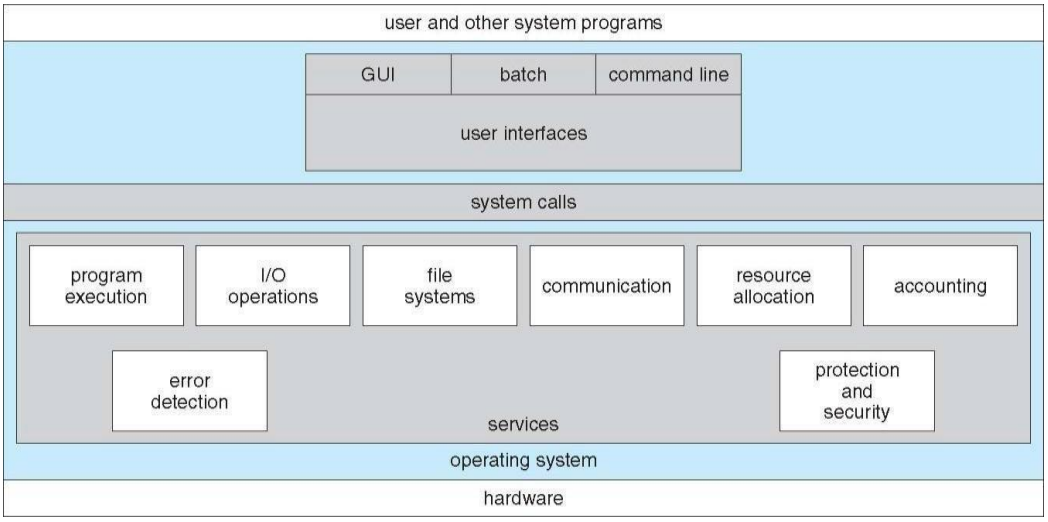


Figure 2.1: Operating System Service

2. **Program Execution** - The OS must be able to load a program into RAM, run the program, and terminate the program, either normally or abnormally (indicating errors).

3. **I/O Operations** - The OS is responsible for transferring data to and from I/O devices, including keyboards, terminals, printers, and files. For specific devices, special functions are provided (device drivers) by OS.

4. **File-System Manipulation** – Programs need to read and write files or directories. They also need to create or delete files, search for a file, list the contents of a file and change the file permissions are provided by OS.
    - Some programs include permissions management to allow or deny access to files or directories based on file ownership.

5. **Communications** – There are many circumstances in which one process needs to exchange information with another process.
    - Such communication may occur between processes that are executing on same computer or between processes that are executing on different computers that are tied together by a computer network.
    - Communications may be implemented by using the service of OS- like shared memory or message passing.

6. **Error Detection** - Both hardware and software errors must be detected and handled appropriately by the OS.
    - Errors may occur - in the CPU and memory hardware (such as power failure and memory error),
        i. in I/O devices (such as a parity error on tape, a connection failure on a network, or lack of paper in the printer), and
        ii. in the user program (such as an arithmetic overflow, an attempt to access an illegal memory location).
    - For each type of error, the OS should take the appropriate action to ensure correct and consistent computing.

- **OS provide services for the efficient operation of the system**, including:

i. **Resource Allocation** – when there are multiple users or multiple jobs running at the same time, resources like CPU cycles, main memory, storage space, and I/O devices mustbe allocated to each of them.

ii. **Accounting** – we want to keep track of which users use how much and what kinds of computer resources. This record keeping may be used either for billing purposes or for statistical record keeping that can be used to optimize future performance.

iii. **Protection and Security** – The owners of information (file) stored in multiuser or networked computer system may want to control the use of that information.
    - When several separate processes execute concurrently, one process should not interfere with other or with OS.

- Protection involves ensuring that all access to system resources is controlled.
- Security of the system from outsiders must also be done, by means of a password to gains access to system resources.

2.2 User Operating-System Interface:

- There are several ways for users to interface with the operating system.

  1) **Command-line interface,** or **command interpreter**, allows users to directly enter commands to be performed by the operating system.

  2) **Graphical user interface (GUI),** allows users to interface with the operating system using pointer device and menu system.

2.3 System Calls:

- System calls provide an interface to the services made available by an OS. i.e., system calls is a means to access the services of the OS.
- Generally written in C or C++, although some are written in assembly for optimal performance.
- The below figure illustrates the sequence of system calls required to copy a file content from one file (input file) to another file (output file). (Figure 2.2)

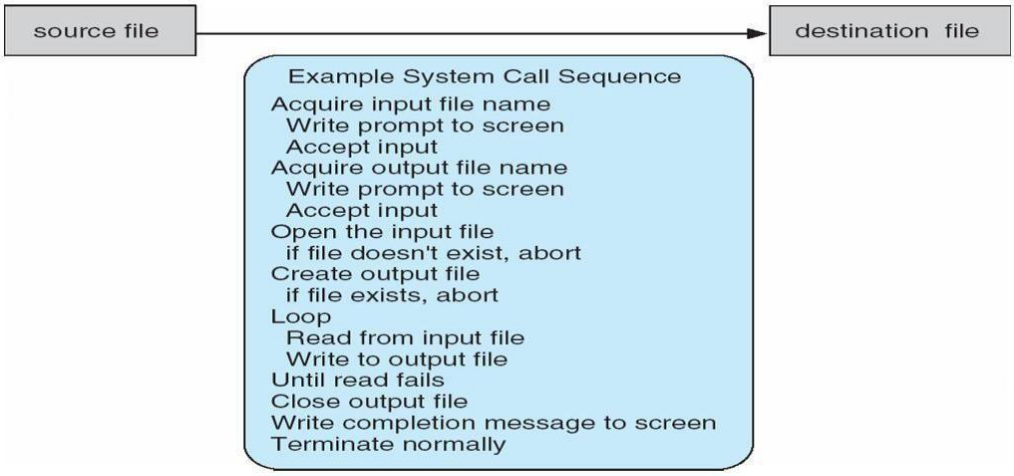


Figure 2.2: Example of how system calls are used.

- **There are number of system calls used to finish this task.**

  1. The first system call is to write a message on the screen (monitor). Then to accept the input filename.

  2. Then another system call to write message on the screen, then to accept the output filename.

3. When the program tries to open the input file, it may find that there is no file of that name or that the file is protected against access.

- In these cases, the program should print a message on the console (another system call) and then terminate abnormally (another system call) and create a new one (another system call).
- Now that both the file as are opened, we enter a loop that reads from the input file (another system call) and writes to output file (another system call).

4. Finally, after the entire file is copied, the program may close both files (another system call), write a message to the console or window (system call), and finally terminate normally (final system call).

➢ **API – System Call – OS Relationship:**

- Most programmers do not use the low-level system calls directly, but instead use an "**Application Programming Interface**", **API**.
- The APIs instead of direct system calls provides for greater program portability between different systems.
- The API then makes the appropriate system calls through the system call interface, using a system call table to access specific numbered system calls, as shown in Figure 2.3.
- Each system call has a specific numbered system call. The system call table (consisting of system call number and address of the particular service) invokes a particular service routine for a specific system call.
- The caller need know nothing about how the system call is implemented or what it does during execution.

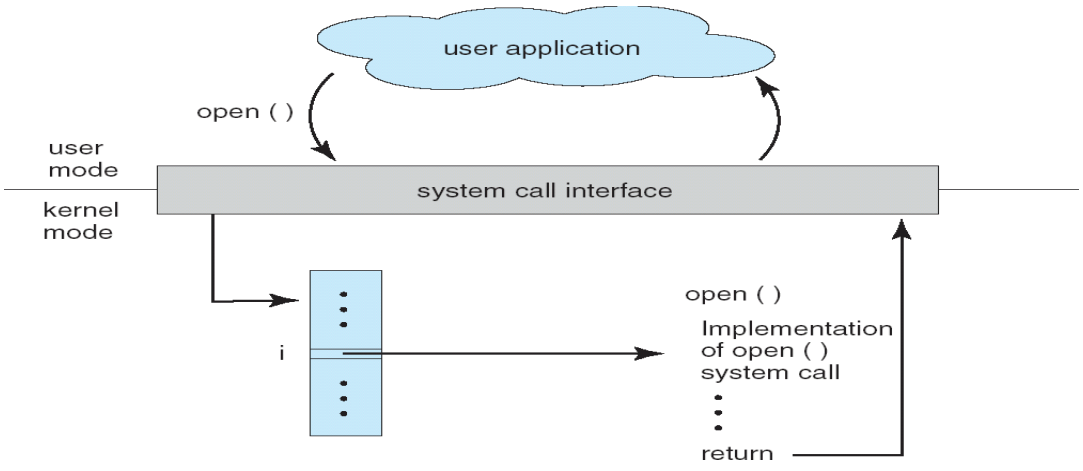


Figure 2.3: The handling of a user application invoking the open( ) system call.

- **Three general methods used to pass parameters to OS are:**

1. To pass parameters in registers.
2. If parameters are large blocks, address of block (where parameters are stored in memory) is sent to OS in the register. (Linux & Solaris).
3. Parameters can be pushed onto the stack by program and popped off the stack by OS.

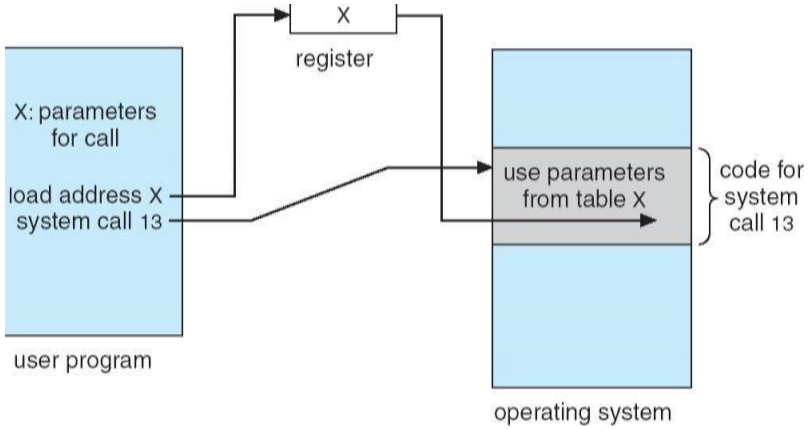


Figure 2.4: Parameter Passing via Table

2.4 Types of System Calls:

- The **system calls** can be categorized into **five major categories:**

1. Process Control
2. File management
3. Device management
4. Information management
5. Communications

- Process control
  - end, abort
  - load, execute
  - create process, terminate process
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
- File management
  - create file, delete file
  - open, close
  - read, write, reposition
  - get file attributes, set file attributes
- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices
- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get process, file, or device attributes
  - set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages
  - transfer status information
  - attach or detach remote devices

Figure 2.4: Types of system calls

2.4.1 Process Control:

- **Process control system calls include** end, abort, load, execute, create process, terminate process, get/set process attributes, wait for time or event, signal event, and allocate and free memory.
- Processes must be created, launched, monitored, paused, resumed, and eventually stopped.
- When one process pauses or stops, then another must be launched or resumed
- Process attributes like process priority, maximum allowable execution time etc. are set and retrieved by OS.
- After creating the new process, the parent process may have to wait (wait time), or wait for an event to occur (wait event). The process sends back a signal when the event has occurred (signal event).

- In DOS, the command interpreter loaded first. Then loads the process and transfers control to it. The interpreter does not resume until the process has completed, as shown in Figure 2.6:

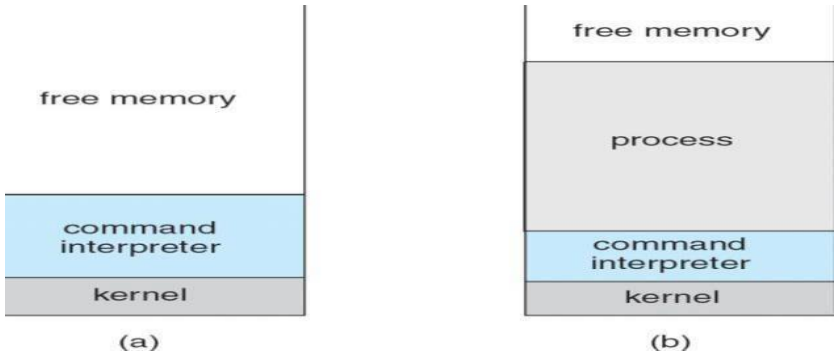


Figure 2.5: MS-DOS execution. (a) At system startup (b) running a program

2.4.2 File Management :

- **The file management functions of OS are** –
- File management system calls include create file, delete file, open, close, read, write, reposition, get file attributes, and set file attributes.
- After creating a file, the file is opened. Data is read or written to a file.
- The file pointer may need to be repositioned to a point.
- The file attributes like filename, file type, permissions, etc. are set and retrieved using system calls.
- These operations may also be supported for directories as well as ordinary files.

2.4.3 Device Management:

- **Device management system calls** include request device, release device, read, write, reposition, get/set device attributes, and logically attach or detach devices.
- When a process needs a resource, a request for resource is done. Then the control is granted to the process. If requested resource is already attached to some other process, the requesting process has to wait.
- In multiprogramming systems, after a process uses the device, it has to be returned to OS, so that another process can use the device.
- Devices may be physical (e.g. disk drives), or virtual / abstract (e.g. files, partitions, and RAM disks).

2.4.4 Information Maintenance:

- Information maintenance system calls include calls to get/set the time, date, system data, and process, file, or device attributes.
- These system calls is used to transfer the information between user and the OS. Information like current time & date, no. of current users, version no. of OS, amount of free memory, disk space etc., are passed from OS to the user.

2.4.5 Communication:

- Communication system calls create/delete communication connection, send/receive messages, transfer status information, and attach/detach remote devices.
- The message passing model must support calls to:
  - Identify a remote process and/or host with which to communicate.
  - Establish a connection between the two processes.
  - Open and close the connection as needed.
  - Transmit messages along the connection.
  - Wait for incoming messages, in either a blocking or non-blocking state.
  - Delete the connection when no longer needed.
- The shared memory model must support calls to:
  - Create and access memory that is shared amongst processes (and threads).
  - Free up shared memory and/or dynamically allocate it as needed.
- Message passing is simpler and easier, (particularly for inter-computer communications), and is generally appropriate for small amounts of data. It is easy to implement, but there are system calls for each read and write process.
- Shared memory is faster, and is generally the better approach where large amounts of data are to be shared. This model is difficult to implement, and it consists of only few system calls.

2.5 System Programs:

- A collection of OS programs that provide a convenient environment for program development and execution (other than OS) are called system programs or system utilities.
- It is not a part of the kernel or command interpreters.

- System programs may be divided into **six categories**:
  - **File management** – These programs create, delete, copy, rename, print, list, and generally manipulate files and directories.
  - **Status information** – Some programs check on the date, time, number of users, processes running, data logging, etc. System registries are used to store and recall configuration information for particular applications.
  - **File modification** – Several text editors and other tools can modify the content of files stored on disk or other storage devices.
  - **Programming-language support** - E.g. Compilers, linkers, debuggers, profilers, assemblers, library archive management, interpreters for common languages, are provided to user with the OS.
  - **Program loading and execution** – Once the program is assembled or complied it must be loaded into memory for execution. The system may provide loaders, dynamic loaders, overlay loaders, etc., as well as interactive debuggers.
  - **Communications** - Programs for providing connectivity between processes and users, including mail, web browsers, remote logins, file transfers, and remote command execution.

2.6 Operating-System Design and Implementation:

2.6.1 Design Goals:

- Any system to be designed must have its own goals and specifications. Similarly the OS to be built will have its own goals depending on the type of system in which it will be used, the type of hardware used in the system etc.
- Requirements define properties which the finished system must have, and are a necessary step in designing any large complex system.
- **The requirements may be of two basic groups**:

1. User goals (User requirements)
2. System goals (system requirements)

- User requirements are features that users care about and understand like system should be convenient to use, easy to learn, reliable, safe and fast.
- System requirements are written for the developers, i.e., People who design the OS. Their requirements are like easy to design, implement and maintain, flexible, reliable, error free and efficient.

2.6.2 Mechanisms and Policies:

- Policies determine what is to be done.
- Mechanisms determine how it is to be implemented.
- **Example**: in timer, counter and decrementing counter is the mechanism and deciding how long the time has to be set is the policies.
- Policies change overtime. In the worst case, each change in policy would require a change in the underlying mechanism.
- If properly separated and implemented, policy changes can be easily adjusted without re-writing the code, just by adjusting parameters or possibly loading new data / configuration files.

2.6.3 Implementation:

- Traditionally OS were written in assembly language.
- In recent years, OS are written in C, or C++. Critical sections of code are still written in assembly language.
- The first OS that was not written in assembly language was the Master Control Program (MCP).
- The advantages of using a higher-level language for implementing operating systems are: The code can be written faster, more compact, easy to port to other systems and is easier to understand and debug.
- The only disadvantages of implementing an operating system in a higher-level language are reduced speed and increased storage requirements.

2.7 Operating-System Structure:

- OS structure must be carefully designed. The task of OS is divided into small components and then interfaced to work together.

2.7.1 Simple Structure:

- Many operating systems do not have well-defined structures.
- They started as small, simple, and limited systems and then grew beyond their original scope, e.g.: MS-DOS.
- In MS-DOS, the interfaces and levels of functionality are not well separated.
- Application programs can access basic I/O routines to write directly to the display and disk drives.
- Such freedom leaves MS-DOS in bad state and the entire system can crash down when user programs fail.

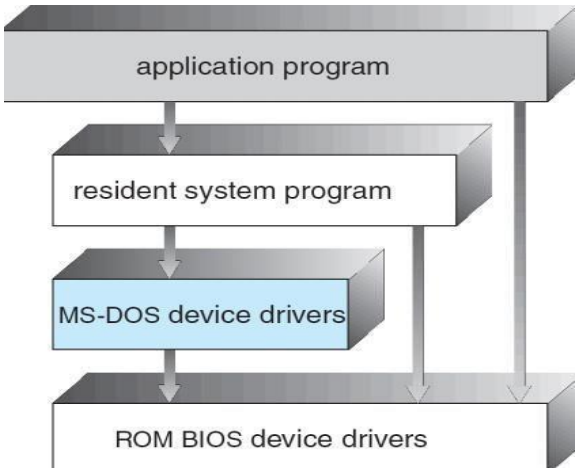


Figure 2.7: MS-DOS Layer Structure

- UNIX OS consists of two separable parts: the kernel and the system programs.
- The kernel is further separated into a series of interfaces and device drivers.
- The kernel provides the file system, CPU scheduling, memory management, and other operating-system functions through system calls.
- This monolithic structure is difficult to implement and maintain.

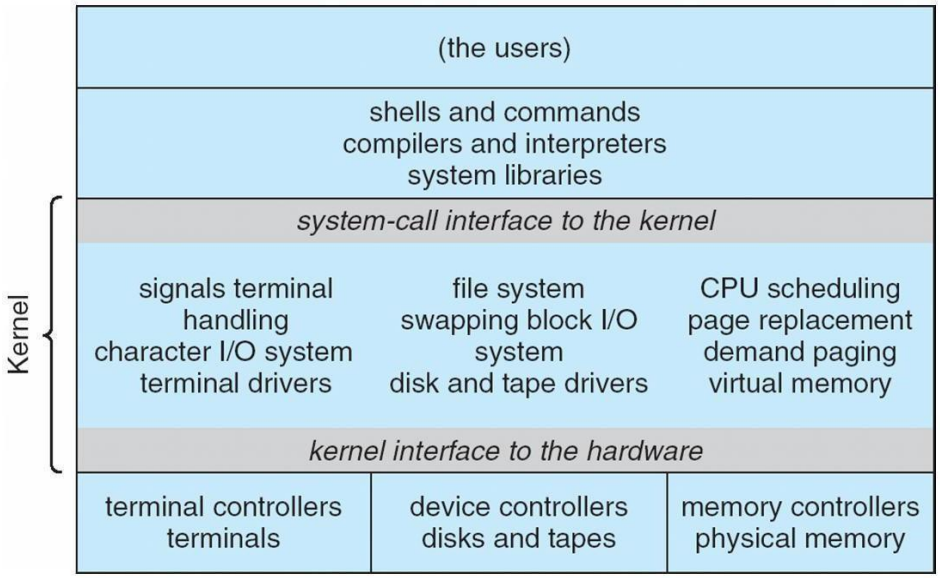


Figure 2.8: UNIX system structure

2.7.2 Layered Approach:

- The OS is broken into number of layers (levels). Each layer rests on the layer below it, and relies on the services provided by the next lower layer.
- Bottom layer (layer 0) is the hardware and the topmost layer is the user interface.
- A typical OS layer say M, consists of data structure and routines that can be invoked by higher-level layer.
- Layer M, in turn, can invoke operations on lower-level layer.

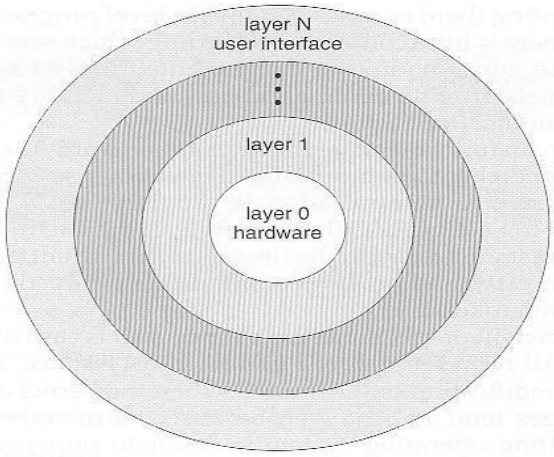


Figure 2.9: A layered Operating System

- **Advantage of layered approach** is simplicity of construction and debugging.
  - The layers are selected so that each uses functions and services of only lower-level layers. So simplifies **debugging and system verification**.
  - The layers are debugged one by one from the lowest and if any layer doesn't work, then error is due to that layer only, as the lower layers are already debugged. Thus the **design and implementation** is simplified.
  - A layer need not know how its lower level layers are implemented. Thus **hides the operations from higher layers.**
- **Disadvantages of layered approach:**
  - The various layers must be appropriately defined, as a layer can use only lower level layers.
  - Less efficient than other types, because any interaction with layer 0 required from top layer.
  - The system call should pass through all the layers and finally to layer 0. This is an overhead.

2.7.3 Microkernels:

- The basic idea behind micro kernels is to remove all non-essential services from the kernel, thus making the kernel as small and efficient as possible.
- The removed services are implemented as **system applications**.
- Most microkernels provide basic process and memory management, and message passing between other services.
- **Benefit of microkernel** - System expansion can also be easier, because it only involves adding more system applications, not rebuilding a new kernel.
- **Mach** was the first and most widely known microkernel, and now forms a major component of Mac OS X.
- **Disadvantage of Microkernel** is, it suffers from reduction in performance due to increases system function overhead.

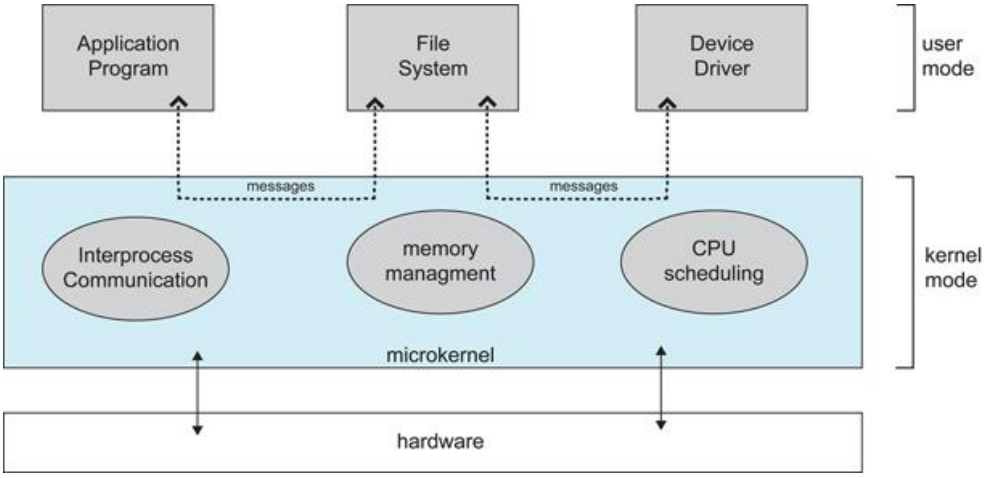


Figure 2.10 Architecture of a typical Microkernel

2.7.4 Modules:

- Modern OS development is object-oriented, with a relatively small core kernel and a set of modules which can be linked in dynamically.
- Modules are similar to layers in that each subsystem has clearly defined tasks and interfaces, but any module is free to contact any other module, eliminating the problems of going through multiple intermediary layers.
- The kernel is relatively small in this architecture, similar to microkernels, but the kernel does not have to implement message passing since modules are free to contact each other directly. Eg: Solaris, Linux and Mac OS X.
- For example, the Solaris OS structure is organized around a core kernel with seven types of loadable kernel modules:

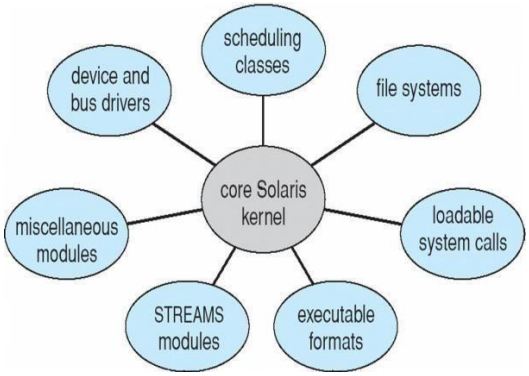


Figure 2.10: Solaris loadable module

1. Scheduling classes
2. File systems
3. Loadable system calls
4. Executable formats
5. STREAMS modules
6. Miscellaneous
7. Device and bus drivers

- Such a design allows the kernel to provide core services yet allows certain features to be implemented dynamically.
- The **Mac OS X architecture** relies on the Mach microkernel for basic system management services, and the BSD kernel for additional services. Application services and dynamically loadable modules (kernel extensions) provide the rest of the OS functionality.
- Resembles layered system, but a module can call any other module.
- Resembles microkernel, the primary module has only core functions and the knowledge of how to load and communicate with other modules.

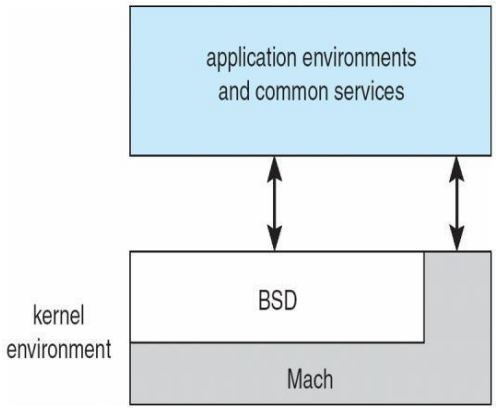


Figure 2.11 The Mac OS X Structure.